MultiAgent-Unreal 架构设计

1. 系统概述

MultiAgent-Unreal 是一个基于 Unreal Engine 5 的多智能体仿真框架，用于机器人、无人机、角色等异构智能体的协同仿真。

2. 核心架构：CARLA 风格 + State Tree + GAS

2.1 CARLA 风格设计

本项目采用 **CARLA 风格架构**，参数存储在实体上而非外部配置：

```
┌──────────────────────────────────────────────────────────────┐
│                    CARLA 风格参数管理                         │
│                                                               │
│  Robot (实体)                    Task/Ability (行为)          │
│  ┌──────────────────┐            ┌──────────────────┐         │
│  │ ScanRadius=200   │◄───────────│ MA Follow        │         │
│  │ FollowTarget     │  获取参数  │ MA Coverage      │         │
│  │ CoverageArea     │◄───────────│ MA Patrol        │         │
│  │ PatrolPath       │            └──────────────────┘         │
│  └──────────────────┘                                         │
└──────────────────────────────────────────────────────────────┘
```

核心原则：

- 参数存储在 Robot 上（如 `ScanRadius`, `FollowTarget`）
- Task/Ability 从 Robot 获取参数，不自己存储
- 同一参数可被多个 Task/Ability 复用
- 运行时可动态修改

Robot 共用参数：

属性	类型	用途
`ScanRadius`	float	Follow 距离、Coverage 间距、Patrol 到达判定
`FollowTarget`	AMACharacter*	跟随目标
`CoverageArea`	AActor*	覆盖区域
`PatrolPath`	AMAPatrolPath*	巡逻路径

属性	类型	用途
`ChargeRate`	float	充电速率（每秒恢复百分比，默认 20%）

2.2 State Tree + GAS 黄金搭档

本项目采用 **State Tree + GAS** 黄金搭档架构：

```
State Tree (大脑 - 状态决策)          GAS (手脚 - 技能执行)
┌──────────────────────────┐          ┌──────────────────────────┐
│  Root                    │          │  Abilities               │
│  ├── Exploration State   │◄────────►│  ├── GA_Pickup           │
│  ├── PhotoMode State     │  Tags    │  ├── GA_Drop             │
│  └── Interaction State   │◄────────►│  ├── GA_Navigate         │
│      ├── Pickup          │          │  └── GA_Follow           │
│      └── Dialogue        │          │                          │
└──────────────────────────┘          └──────────────────────────┘
```

- **State Tree**: 高层策略决策，管理 Character 的主模式（探索/拍照/交互）
- **GAS**: 具体技能执行，处理动画、冷却、消耗等
- **Gameplay Tags**: 两者之间的通信桥梁

3. 当前文件结构

```
MultiAgent-Unreal/
├── config/                          # 配置文件目录
│   └── agents.json                  # Agent 配置 (JSON 驱动创建)
│
└── unreal_project/Source/MultiAgent/
    ├── Core/                        # 核心框架
    │   ├── MATypes.h                # 公共类型定义 (EMAAgentType, EMAFormationType)
    │   ├── MAAgentManager.h/cpp     # Agent 管理器 (JSON 配置 + Action 动态发现)
    │   ├── MAGameMode.h/cpp         # 游戏模式 (配置驱动)
    │   └── MAPlayerController.h/cpp # 玩家控制器 (Enhanced Input)
    │
    ├── Agent/                       # 智能体模块
    │   ├── Character/               # ACharacter 派生类
    │   │   ├── MACharacter.h/cpp    # 角色基类 (GAS + Sensor + Action 聚合)
    │   │   ├── MAHumanCharacter.h/cpp   # 人类角色
    │   │   └── MARobotDogCharacter.h/cpp # 机器狗角色
    │   │
    │   ├── Component/               # 组件模块 (UE Component 模式)
    │   │   └── Sensor/              # 传感器组件
    │   │       ├── MASensorComponent.h/cpp       # 传感器基类 (Action 接口)
    │   │       └── MACameraSensorComponent.h/cpp # 摄像头组件 (6 个 Actions)
│   │
    │   ├── GAS/                     # GAS 模块
```

```
    │   │   ├── MAAbilitySystemComponent.h/cpp  # GAS 核心组件
    │   │   ├── MAGameplayTags.h/cpp # Gameplay Tags 定义
    │   │   ├── MAGameplayAbilityBase.h/cpp  # Ability 基类
    │   │   └── Ability/             # 具体技能
    │   │       ├── GA_Pickup.h/cpp  # 拾取技能
    │   │       ├── GA_Drop.h/cpp    # 放下技能
    │   │       ├── GA_Navigate.h/cpp # 导航技能
    │   │       ├── GA_Follow.h/cpp  # 追踪技能
    │   │       ├── GA_Search.h/cpp  # 搜索技能
    │   │       ├── GA_Observe.h/cpp # 观察技能
    │   │       ├── GA_Report.h/cpp  # 报告技能
    │   │       ├── GA_Charge.h/cpp  # 充电技能
    │   │       ├── GA_Formation.h/cpp # 编队技能
    │   │       └── GA_Avoid.h/cpp   # 避障技能
    │   │
    │   └── StateTree/               # State Tree 模块
    │       ├── MAStateTreeComponent.h/cpp   # State Tree 组件
    │       ├── Task/                # State Tree Task
    │       │   ├── MASTTask_ActivateAbility.h/cpp
    │       │   ├── MASTTask_Navigate.h/cpp
    │       │   ├── MASTTask_Patrol.h/cpp
    │       │   ├── MASTTask_Charge.h/cpp
    │       │   ├── MASTTask_Coverage.h/cpp
    │       │   └── MASTTask_Follow.h/cpp
    │       └── Condition/           # State Tree Condition
    │           ├── MASTCondition_HasCommand.h/cpp
    │           ├── MASTCondition_LowEnergy.h/cpp
    │           ├── MASTCondition_FullEnergy.h/cpp
    │           └── MASTCondition_HasPatrolPath.h/cpp
    │
    ├── Environment/                 # 环境 Actor
    │   ├── MAPickupItem.h/cpp       # 可拾取物品
    │   ├── MAChargingStation.h/cpp  # 充电站
    │   ├── MAPatrolPath.h/cpp       # 巡逻路径
    │   └── MACoverageArea.h/cpp     # 覆盖区域
    │
    ├── Input/                       # 输入系统
    │   ├── MAInputActions.h/cpp     # Input Actions 单例 (运行时创建)
    │   ├── MAInputConfig.h/cpp      # 输入配置数据资产
    │   ├── MAInputComponent.h/cpp   # 增强输入组件
    │   └── MAPlayerController.h/cpp # 玩家控制器
    │
    ├── MultiAgent.Build.cs          # 模块构建配置 (含 Json 模块)
    └── MultiAgent.h/cpp             # 模块定义
```

4. 类继承关系

```
ACharacter (UE) + IAbilitySystemInterface
    └── AMACharacter (角色基类, GAS + Sensor + Action 聚合)
            ├── AMAHumanCharacter (人类)
            └── AMARobotDogCharacter (机器狗)
```

```
USceneComponent (UE)
    └── UMASensorComponent (传感器基类，Action 接口)
            └── UMACameraSensorComponent (摄像头组件，6 个 Actions)

AActor (UE)
    ├── AMAPickupItem (可拾取物品)
    ├── AMAChargingStation (充电站)
    └── AMAPatrolPath (巡逻路径)

UAbilitySystemComponent (UE GAS)
    └── UMAAbilitySystemComponent

UGameplayAbility (UE GAS)
    └── UMAGameplayAbilityBase
            ├── UGA_Pickup
            ├── UGA_Drop
            ├── UGA_Navigate
            ├── UGA_Follow
            ├── UGA_Search
            ├── UGA_Observe
            ├── UGA_Report
            ├── UGA_Charge
            ├── UGA_Formation
            └── UGA_Avoid

UWorldSubsystem (UE)
    └── UMAAgentManager (Agent 管理 + JSON 配置 + Action 路由)
```

5. Sensor Component 设计

5.1 从 Actor 到 Component 的重构

旧架构 (Actor 模式):

```
AMASensor (Actor)
    └── AMACameraSensor (Actor)
        - 独立生命周期
        - 需要 AttachToActor()
        - MAActorSubsystem 管理
```

新架构 (Component 模式):

```
UMASensorComponent (USceneComponent)
    └── UMACameraSensorComponent
        - 作为 Character 的组件
        - 生命周期由 Character 管理
        - 通过 Character API 操作
```

5.2 设计优势

对比项	Actor 模式	Component 模式
生命周期	独立管理，需手动销毁	随 Character 自动销毁
附着关系	运行时 AttachToActor	构造时 SetupAttachment
管理方式	MAActorSubsystem	Character 自己管理
UE 惯例	非标准	符合 UE Component 模式
代码复杂度	高	低

5.3 MACharacter Sensor 管理 API

```
// 添加相机传感器
UMACameraSensorComponent* Camera = Character->AddCameraSensor(
    FVector(-200, 0, 100),      // 相对位置
    FRotator(-10, 0, 0)         // 相对旋转
);

// 获取相机传感器
UMACameraSensorComponent* Camera = Character->GetCameraSensor();

// 获取所有传感器
TArray<UMASensorComponent*> Sensors = Character->GetAllSensors();

// 移除传感器
Character->RemoveSensor(Sensor);

// 获取传感器数量
int32 Count = Character->GetSensorCount();
```

5.4 MACameraSensorComponent 功能

方法	按键	说明
`TakePhoto(FilePath)`	L	拍照保存到 Saved/Screenshots/
`StartRecording(FPS)`	R	开始录像（序列帧）
`StopRecording()`	R	停止录像
`StartTCPStream(Port, FPS)`	V	启动 TCP 视频流
`StopTCPStream()`	V	停止 TCP 视频流

方法	按键	说明
`CaptureFrame()`	-	获取当前帧数据

属性:

属性	默认值	说明
`Resolution`	640x480	分辨率
`FOV`	90.0	视野角度
`JPEGQuality`	50	JPEG 压缩质量 (10-100)
`StreamFPS`	15.0	流/录像帧率 (5-30)

6. JSON 配置驱动系统

6.1 配置文件结构

Agent 创建由 `config/agents.json` 驱动，无需修改 C++ 代码：

```
{
  "version": "1.0",
  "spawn_settings": {
    "default_spacing": 150.0,
    "auto_position_enabled": true
  },
  "agents": [
    {
      "id": "human_01",
      "type": "Human",
      "class": "/Game/Blueprints/BP_MAHumanCharacter.BP_MAHumanCharacter_C",
      "position": "auto",
      "rotation": { "pitch": 0, "yaw": 0, "roll": 0 },
      "sensors": [
        {
          "type": "Camera",
          "position": { "x": -200, "y": 0, "z": 100 },
          "rotation": { "pitch": -10, "yaw": 0, "roll": 0 },
          "params": { "resolution": { "x": 640, "y": 480 }, "fov": 90 }
        }
      ]
    },
    {
      "id": "dog_01",
      "type": "RobotDog",
      "class": "/Game/Blueprints/BP_MARobotDogCharacter.BP_MARobotDogCharacter_C",
      "position": { "x": 500, "y": 200, "z": 100 }
    }
  ]
```

```
}
```

6.2 配置字段说明

字段	类型	说明
`id`	string	Agent 唯一标识 (FString)
`type`	string	Agent 类型: Human, RobotDog, Drone, Camera
`class`	string	Blueprint 类路径
`position`	object/"auto"	生成位置，"auto" 自动计算
`rotation`	object	生成旋转 (pitch/yaw/roll)
`sensors`	array	传感器配置列表

6.3 加载流程

```
MAGameMode::BeginPlay()
    │
    ▼
MAAgentManager::LoadAndSpawnFromConfig(path)
    │
    ├── 解析 JSON 配置
    ├── 遍历 agents[]
    │   ├── 加载 Blueprint 类
    │   ├── 计算生成位置 (auto 或指定)
    │   ├── SpawnAgent()
    │   └── 添加 Sensors
    └── 返回成功/失败
```

7. Action 动态发现机制

7.1 设计目标

- **可扩展**: 新增 Sensor 自动暴露 Actions，无需修改上层代码
- **统一接口**: 所有 Action 通过 `ExecuteAction(name, params)` 执行
- **自描述**: `GetAvailableActions()` 返回所有可用 Actions

7.2 Action 命名规范

```
<Category>.<ActionName>

Agent.NavigateTo       # Agent 自身 Action
Agent.Pickup
Camera.TakePhoto       # Camera Sensor Action
Camera.StartRecording
Lidar.StartScan        # 未来 Lidar Sensor Action
```

7.3 接口定义

MASensorComponent (基类):

```
// 返回该 Sensor 支持的所有 Actions
virtual TArray<FString> GetAvailableActions() const;

// 执行指定 Action
virtual bool ExecuteAction(const FString& ActionName, const TMap<FString, FString>& Params);
```

MACharacter (聚合层):

```
// 聚合自身 + 所有 Sensor 的 Actions
TArray<FString> GetAvailableActions() const;

// 路由到正确的执行者
bool ExecuteAction(const FString& ActionName, const TMap<FString, FString>& Params);
```

MAAgentManager (管理层):

```
// 获取指定 Agent 的所有 Actions
TArray<FString> GetAgentAvailableActions(AMACharacter* Agent) const;

// 执行指定 Agent 的 Action
bool ExecuteAgentAction(AMACharacter* Agent, const FString& ActionName, const TMap<FString,
FString>& Params);

// 获取所有 Agent 的 Actions 汇总 (C++ only)
TMap<FString, TArray<FString>> GetAllAgentsActions() const;
```

7.4 已实现的 Actions

Agent Actions (MACharacter):

Action	参数	说明
`Agent.NavigateTo`	x, y, z	导航到指定位置
`Agent.CancelNavigation`	-	取消当前导航
`Agent.Pickup`	-	拾取附近物品
`Agent.Drop`	-	放下持有物品
`Agent.FollowActor`	target_id	跟随指定 Agent
`Agent.StopFollowing`	-	停止跟随

Camera Actions (MACameraSensorComponent):

Action	参数	说明
`Camera.TakePhoto`	filename (可选)	拍照保存
`Camera.StartRecording`	fps (可选)	开始录像
`Camera.StopRecording`	-	停止录像
`Camera.StartTCPStream`	port, fps	启动 TCP 流
`Camera.StopTCPStream`	-	停止 TCP 流
`Camera.GetFrameAsJPEG`	-	获取当前帧 JPEG

7.5 扩展示例

添加新 Sensor 只需：

```
// MALidarSensorComponent.h
class UMALidarSensorComponent : public UMASensorComponent
{
    virtual TArray<FString> GetAvailableActions() const override
    {
        return { TEXT("Lidar.StartScan"), TEXT("Lidar.StopScan"), TEXT("Lidar.GetPointCloud") };
    }

    virtual bool ExecuteAction(const FString& ActionName, const TMap<FString, FString>& Params)
override
    {
        if (ActionName == TEXT("Lidar.StartScan")) { StartScan(); return true; }
        // ...
```

```
    }
};
```

Character 自动聚合新 Sensor 的 Actions，无需修改任何代码。

8. Manager 子系统设计

8.1 全局 Manager vs Character 组件

```
┌──────────────────────────────────────────────────────────────┐
│                    全局 Manager 层                            │
│                  UWorldSubsystem                              │
│                                                               │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐        │
│  │AgentManager │  │RelationMgr  │  │ MapManager  │        │
│  │ JSON 配置   │  │ 管理关系    │  │ 管理地图    │        │
│  │ Action 路由 │  │ 之间的关系  │  │ 导航感知    │        │
│  └──────────────┘  └──────────────┘  └──────────────┘        │
└──────────────────────────────────────────────────────────────┘
                            │
                            │ 管理
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                    Character 组件层                           │
│                  每个 Character 独立拥有                       │
│                                                               │
│   Character #1          Character #2          Character #3    │
│  ┌────────────┐         ┌────────────┐         ┌────────────┐ │
│  │ StateTree │         │ StateTree │         │ StateTree │ │
│  │ 决策      │         │ 决策      │         │ 决策      │ │
│  ├────────────┤         ├────────────┤         ├────────────┤ │
│  │ ASC       │         │ ASC       │         │ ASC       │ │
│  │ 技能执行  │         │ 技能执行  │         │ 技能执行  │ │
│  ├────────────┤         ├────────────┤         ├────────────┤ │
│  │ Sensors   │         │ Sensors   │         │ Sensors   │ │
│  │ 传感器组件│         │ 传感器组件│         │ 传感器组件│ │
│  └────────────┘         └────────────┘         └────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

8.2 Manager 列表

Manager	层级	功能介绍	状态
MAAgentManager	全局	JSON 配置加载 + Agent 生命周期 + Action 路由 + 编队管理	✅ 已实现
RelationManager	全局	实体关系管理	❌ 待开发

Manager	层级	功能介绍	状态
MapManager	全局	地图感知	❌ 待开发
StateTree	Character级	状态决策	✅ 已实现
ASC	Character级	技能执行	✅ 已实现
Sensors	Character级	传感器组件 (Action 接口)	✅ 已实现

8.3 编队管理 (Formation Manager)

MAAgentManager 同时作为集群管理器，类似 CARLA 的 TrafficManager：

```
┌──────────────────────────────────────────────────┐
│              MAAgentManager                      │
│  ┌──────────────────┐  ┌──────────────────────────┐  │
│  │  Agent Factory   │  │  Formation Manager       │  │
│  │  - LoadConfig    │  │  - Leader                │  │
│  │  - SpawnAgent    │  │  - Members[]             │  │
│  │  - Query         │  │  - Type                  │  │
│  ├──────────────────┤  │  - CalculatePositions()  │  │
│  │  Action Router   │  │  - UpdateFormation()     │  │
│  │  - GetActions    │  └──────────────────────────┘  │
│  │  - ExecuteAction│                                  │
│  └──────────────────┘                                  │
└──────────────────────────────────────────────────┘
```

编队类型 (EMAFormationType):

类型	说明
None	无编队
Line	横向一字排开
Column	纵队
Wedge	V形楔形
Diamond	菱形
Circle	圆形

7. GAS 模块详解

7.1 Gameplay Tags 定义

```
// State Tags (State Tree 状态)
State.Exploration          // 探索模式
State.PhotoMode            // 拍照模式
State.Interaction          // 交互模式

// Ability Tags (GAS 技能)
Ability.Pickup             // 拾取技能
Ability.Drop               // 放下技能
Ability.Navigate           // 导航技能
Ability.Follow             // 追踪技能
Ability.Search             // 搜索技能
Ability.Observe            // 观察技能
Ability.Report             // 报告技能
Ability.Charge             // 充电技能
Ability.Formation          // 编队技能
Ability.Avoid              // 避障技能

// Event Tags
Event.Target.Found         // 发现目标
Event.Target.Lost          // 丢失目标
Event.Charge.Complete      // 充电完成

// Status Tags
Status.Patrolling          // 正在巡逻
Status.Searching           // 正在搜索
Status.Charging            // 正在充电
Status.InFormation         // 编队中
Status.LowEnergy           // 低电量
```

7.2 Ability 列表

Ability	功能	激活条件
GA_Pickup	拾取物品	附近有可拾取物品
GA_Drop	放下物品	Status.Holding
GA_Navigate	导航移动	有 AIController
GA_Follow	追踪目标	有目标 Character
GA_Search	搜索 Human	有 CameraSensorComponent
GA_Observe	观察目标	-

Ability	功能	激活条件
`GA_Report`	报告信息	-
`GA_Charge`	充电	在充电站范围内
`GA_Formation`	编队移动	有 Leader
`GA_Avoid`	避障	-

8. 输入系统 (Enhanced Input)

8.1 默认按键映射

按键	功能
左键	移动所有 Human Character
右键	移动所有 RobotDog
P	所有 Human 尝试拾取
O	所有 Human 放下物品
Tab	切换 Camera 视角
0	返回上帝视角
F	所有 RobotDog 跟随 Human
G	所有 RobotDog 开始巡逻
H	所有 RobotDog 去充电
J	所有 RobotDog 停止并进入 Idle
L	所有 CameraSensor 拍照
R	开始/停止录像
V	开始/停止 TCP 视频流

9. State Tree 模块详解

9.1 自定义 Task (CARLA 风格)

Task 不存储可配置参数，而是从 Robot 获取：

Task	功能	从 Robot 获取的参数
`MASTTask_Patrol`	循环巡逻路径点	`PatrolPath`, `ScanRadius`
`MASTTask_Navigate`	导航到指定位置	-
`MASTTask_Charge`	自动充电	`ChargeRate`
`MASTTask_Coverage`	区域覆盖扫描	`CoverageArea`, `ScanRadius`
`MASTTask_Follow`	跟随目标	`FollowTarget`, `ScanRadius`

9.2 自定义 Condition

Condition	功能
`MASTCondition_HasCommand`	检查是否收到指定命令
`MASTCondition_LowEnergy`	检查电量 < 阈值
`MASTCondition_FullEnergy`	检查电量 >= 阈值
`MASTCondition_HasPatrolPath`	检查是否有可用巡逻路径

9.3 状态转换示例

```
┌──────────────┐  Command.Patrol  ┌──────────────┐
│     Idle     │ ────────────────► │    Patrol    │
└──────────────┘                  └──────────────┘
       ▲                                  │
       │ Command.Idle                     │ LowEnergy (<20%)
       │                                  ▼
       │                          ┌──────────────┐
       └──────────────────────────│    Charge    │
              FullEnergy (100%)   └──────────────┘
```

12. 开发路线

Phase 1: 核心框架 ✅ 已完成

√ UMAActorSubsystem - Character 生命周期管理

√ AMACharacter 及其子类

√ GAS 集成 - AbilitySystemComponent

√ 所有 Ability (Pickup, Drop, Navigate, Follow, Search, etc.)

√ State Tree 基础集成

Phase 2: Sensor Component 重构 ✅ 已完成

√ Sensor 从 Actor 改为 Component 模式

√ UMASensorComponent 基类

√ UMACameraSensorComponent (拍照/录像/TCP流)

√ MACharacter Sensor 管理 API

√ MAActorSubsystem 移除 Sensor 管理代码

Phase 3: AgentManager + Action 动态发现 ✅ 已完成

√ MAActorSubsystem → MAAgentManager 重命名

√ MATypes.h 公共类型定义 (避免循环依赖)

√ JSON 配置驱动 Agent 创建 (config/agents.json)

√ Action 动态发现机制 (GetAvailableActions + ExecuteAction)

√ MASensorComponent Action 接口

√ MACameraSensorComponent 6 个 Actions

√ MACharacter Action 聚合 + 6 个 Agent Actions

√ 属性重命名: ActorID/Name/Type → AgentID/Name/Type (FString)

Phase 4: 待开发

√ 配置热重载 - 运行时修改 JSON 自动刷新

√ 可视化编辑器 - UE Editor 内编辑 Agent 配置

√ 更多 Sensor 类型 (Lidar, Depth, IMU)

√ 更多 Agent 类型 (Drone)

√ UMARelationSubsystem - 实体关系图

√ SaveGame 系统 - 游戏存档/读档

13. 重要设计原则

13.1 Component 优于 Actor

对于附属于 Character 的功能模块（如 Sensor），优先使用 UE Component 模式：

```
// ✅ 推荐：Component 模式
UMACameraSensorComponent* Camera = Character->AddCameraSensor(Location, Rotation);

// ❌ 不推荐：Actor 模式
AMACameraSensor* Camera = World->SpawnActor<AMACameraSensor>();
Camera->AttachToActor(Character, ...);
```

13.2 CARLA 风格参数管理

参数存储在实体上，行为从实体获取参数：

```
// ✅ 推荐：从 Robot 获取参数
float Distance = Robot->ScanRadius;
AMACharacter* Target = Robot->GetFollowTarget();

// ❌ 不推荐：在 Task/Ability 中硬编码参数
float Distance = 200.f;  // 硬编码
```

13.3 虚函数多态设计

不同类型的 Character 可能需要不同的行为实现：

```
// MACharacter.h (基类)
virtual void OnNavigationTick();

// MAHumanCharacter.cpp (子类重写)
void AMAHumanCharacter::OnNavigationTick()
{
    // Human 特有行为
}
```

13.4 配置驱动优于硬编码

Agent 创建应通过配置文件驱动，而非 C++ 硬编码：

```
// ✅ 推荐：配置驱动
AgentManager->LoadAndSpawnFromConfig("config/agents.json");

// ❌ 不推荐：硬编码
for (int i = 0; i < 3; i++) {
    World->SpawnActor<AMARobotDogCharacter>(...);
}
```

13.5 Action 自描述原则

每个 Sensor/Component 应自描述其支持的 Actions：

```
// ✅ 推荐：自描述 Actions
TArray<FString> GetAvailableActions() const override
{
    return { TEXT("Camera.TakePhoto"), TEXT("Camera.StartRecording") };
}

// ❌ 不推荐：外部维护 Action 列表
// 在某个全局配置中硬编码所有 Actions
```